RTAB-VSLAM 3D Mapping

1. RTAB-VSLAM Description

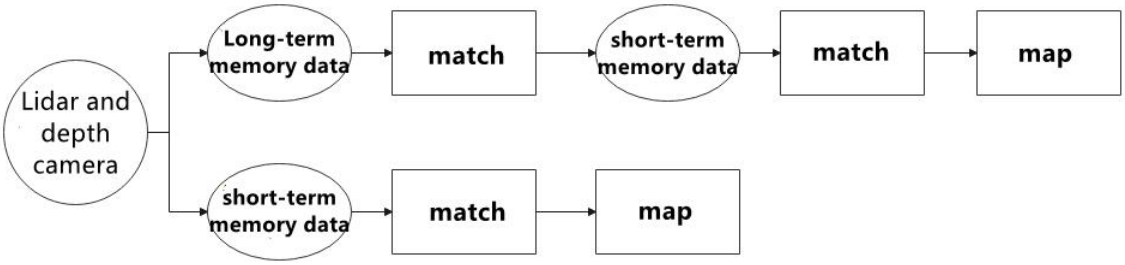
RTAB-VSLAM is a appearance-based real-time 3D mapping system, it's an open-source library that achieves loop closure detection through memory management methods. It limits the size of the map to ensure that loop closure detection is always processed within a fixed time limit, thus meeting the requirements for long-term and large-scale environment online mapping.

2. RTAB-VSLAM Working Principle

RTAB-VSLAM 3D mapping employs feature mapping, offering the advantage of rich feature points in general scenes, good scene adaptability, and the ability to use feature points for localization. However, it has drawbacks, such as a time-consuming feature point calculation method, limited information usage leading to loss of image details, diminished effectiveness in weak-texture areas, and susceptibility to feature point matching errors, impacting results significantly.

After extracting features from images, the algorithm proceeds to match features at different timestamps, leading to loop detection. Upon completion of matching, data is categorized into long-term memory and short-term memory. Long-term memory data is utilized for matching future data, while short-term memory data is employed for matching current time-continuous data.

During the operation of the RTAB-VSLAM algorithm, it initially uses short-term memory data to update positioning points and build maps. As data from a specific future timestamp matches long-term memory data, the corresponding long-term memory data is integrated into short-term memory data for updating positioning and map construction.



RTAB-VSLAM software package link: https://github.com/introlab/rtabmap

3. RTAB-VSLAM 3D Mapping Instructions

3.1 Robot Operations

1) Click on  on the system desktop to open the command-line terminal.

2) Run the command to disable the app auto-start service:

```
~/.stop_ros.sh
```

3) Execute the command to start mapping:

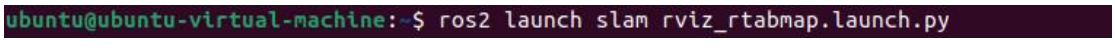
```
ros2 launch slam rtabmap_slam.launch.py
```

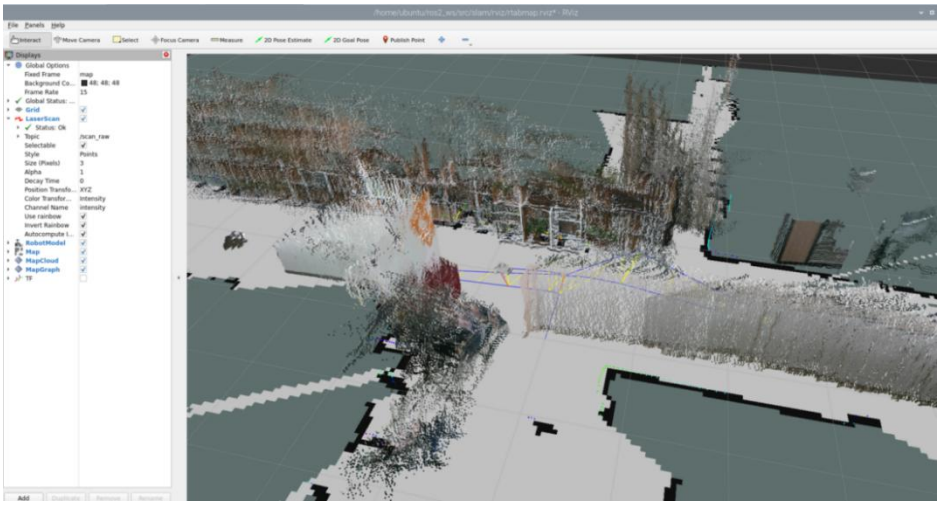
3.2 Virtual Machine Operation

1) Click-on  to open the command-line terminal.

2) Enter the command to open the RViz tool and display the mapping effect:

```
ros2 launch slam rviz_rtabmap.launch.py
```





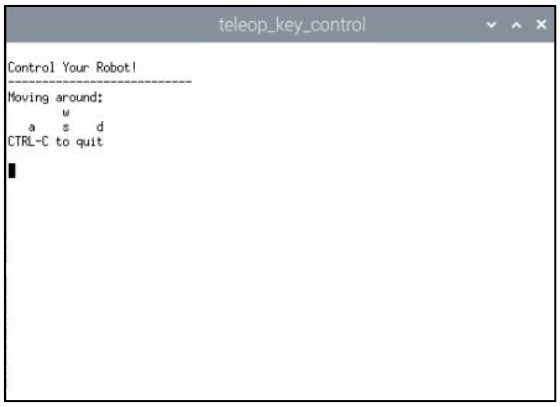
3.3 Enable Keyboard Control

1) Click-on  to open the command-line terminal.

2) Enter the command to start the keyboard control node, and press “Enter”.

```
ros2 launch peripherals teleop_key_control.launch.py
```

If you encounter the prompt as shown in the figure below, it means that the keyboard control service has been successfully activated.



3) Control the robot to move in the current space to build a more complete map. The table below shows the keyboard keys available for controlling robot movement and their corresponding functions:

Key	Robot Action

W	Short press to switch to the forward state and continuously move forward
S	Short press to switch to the backward state and continuously move backward
A	Long press to interrupt the forward or backward state and turn left
D	Long press to interrupt the forward or backward state and turn right

When controlling the robot's movement for mapping using the keyboard, it's advisable to appropriately reduce the robot's movement speed. The smaller the robot's running speed, the smaller the relative error of the odometry, resulting in a better mapping effect. As the robot moves, the map displayed in RVIZ will continuously expand until the entire environmental scene's map construction is completed.

3.4 Map Saving

After mapping is completed, you can use the shortcut "Ctrl+C" in each command-line terminal window to close the currently running program.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

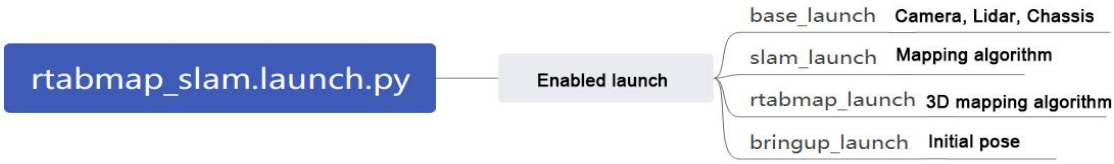
Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```

Note: For 3D mapping, there's no need to manually save the map. When you use "Ctrl+C"

to close the mapping command, the map will be automatically saved.

3.5 Launch File Analysis



The launch file is located at:

/home/ubuntu/ros2_ws/src/slam/launch/rtabmap_slam.launch.py

◆ **Import Library:**

You can refer to the ROS official documentation for detailed analysis of the launch library:

"https://docs.ros.org/en/humble/How-To-Guides/Launching-composable-nodes.html"

```
import os
from ament_index_python.packages import get_package_share_directory
from launch_ros.actions import PushRosNamespace
from launch import LaunchDescription
from launch.substitutions import LaunchConfiguration
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, GroupAction, OpaqueFunction, TimerAction
```

◆ **Set the Storage Path**

Use "get_package_share_directory" to obtain the path of the slam package.

```
    if compiled == 'True':
        slam_package_path = get_package_share_directory('slam')
    else:
        slam_package_path = '/home/ubuntu/ros2_ws/src/slam'
```

◆ **Initiate Other Launch File**

```
    base_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(slam_package_path, 'launch/include/robot.launch.py')),
        launch_arguments={
            'sim': sim,
            'master_name': master_name,
            'robot_name': robot_name,
            'action_name': 'horizontal',
        }.items(),
    )

    slam_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(slam_package_path, 'launch/include/slam_base.launch.py')),
        launch_arguments={
            'use_sim_time': use_sim_time,
            'map_frame': map_frame,
            'odom_frame': odom_frame,
            'base_frame': base_frame,
            'scan_topic': scan_topic,
        }.items(),
    )

    rtabmap_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(slam_package_path, 'launch/include/rtabmap.launch.py')),
        launch_arguments={
            'use_sim_time': use_sim_time,
            'frame_id': base_frame,
            'odom_frame_id': odom_frame,
            'map_frame_id': map_frame,
            'wait_for_transform_duration': '0.2',
            'max_tf_cache_size': '100',
            'max_imu_buffer_size': '100',
        }.items(),
    )
```

base_launch: Launch for hardware initialization

slam_launch: Basic mapping launch

rtabmap_launch: RTAB mapping launch

bringup_launch: Initial pose launch